MYSQL LAB ASSIGNMENT

Tutorials 7 – 10

Joins • Grouping & Ordering • Procedures • Triggers

NAME : Harshitha GM

USN : 1RUA25BCA0040

SEC 'A' sec

Each experiment is presented with the corresponding MySQL input and output screenshot.

Tutorial 7 – Implement Different Types of Joins

Implement Inner Join, Outer Join (LEFT/RIGHT), and Natural Join with suitable examples.

Input & Output – Database/table setup and sample data

```
mysql> USE STUDENT;
ERROR 1049 (42000): Unknown database 'student'
mysql> CREATE DATABASE STUDENT;
Query OK, 1 row affected (0.365 sec)

mysql> USE STUDENT;
Database changed
mysql> CREATE  TABLE Std ( Student_ID INT PRIMARY KEY,Name VARCHAR(30),City
VARCHAR(20));
Query OK, 0 rows affected (1.487 sec)

mysql> DESC Std;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| Student_ID | int         | NO   | PRI | NULL    |       |
| Name       | varchar(30) | YES  |     | NULL    |       |
| City       | varchar(20) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
3 rows in set (0.898 sec)

mysql> INSERT INTO Std VALUES (101,'Harshitha','Bangaluru'),(102,'Chandana',
'Mysuru'),(103,'Monisha','Hubli'),(104,'Gayathri','Bangaluru');
Query OK, 4 rows affected (0.074 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO MARKS VALUES(101,'DBMS',86),(102,'DBMS',90),(103,'DBMS',7
8),(105 ,'DBMS',88);
ERROR 1146 (42S02): Table 'student.marks' doesn't exist
mysql> CREATE  TABLE MARKS ( Student_ID INT,Subject VARCHAR(30),Marks INT );

Query OK, 0 rows affected (0.238 sec)

mysql> INSERT INTO MARKS VALUES(101,'DBMS',86),(102,'DBMS',90),(103,'DBMS',7
8),(105 ,'DBMS',88);
Query OK, 4 rows affected (0.560 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
```

MySQL Lab Assignment – Input & Output

Tutorial 7 – Implement Different Types of Joins

Input & Output – INNER JOIN, LEFT JOIN and RIGHT JOIN

```
|       NULL | NULL       |    88 |
+------------+------------+-------+
4 rows in set (0.010 sec)

mysql> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std INNER JOIN MARK
S  ON Std.Student_ID = MARKS.Student_ID;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
+------------+-----------+-------+
3 rows in set (0.014 sec)

mysql> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std LEFT JOIN MARKS
  ON Std.Student_ID = MARKS.Student_ID;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
|        104 | Gayathri  |  NULL |
+------------+-----------+-------+
4 rows in set (0.009 sec)

mysql> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std RIGHT JOIN MARK
S  ON Std.Student_ID = MARKS.Student_ID;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
|       NULL | NULL      |    88 |
+------------+-----------+-------+
4 rows in set (0.011 sec)

mysql>
```

MySQL Lab Assignment – Input & Output

Tutorial 7 – Implement Different Types of Joins

Input & Output – Combined join result and NATURAL JOIN

```
4 rows in set (0.011 sec)

mysql> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std LEFT JOIN MARKS
  ON Std.Student_ID = MARKS.Student_ID
    -> UNION
    -> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std RIGHT JOIN MARK
S  ON Std.Student_ID = MARKS.Student_ID;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
|        104 | Gayathri  |  NULL |
|       NULL | NULL      |    88 |
+------------+-----------+-------+
5 rows in set (0.299 sec)

mysql> SELECT Student_ID , Name , MARKS
    -> FROM Std
    -> NATURAL JOIN MARKS;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
+------------+-----------+-------+
3 rows in set (0.300 sec)

mysql> SELECT City ,COUNT(*) AS Total_Students
    -> FROM Std
    -> GROUP BY City;
+-----------+----------------+
| City      | Total_Students |
+-----------+----------------+
| Bangaluru |              2 |
| Mysuru    |              1 |
| Hubli     |              1 |
+-----------+----------------+
3 rows in set (0.311 sec)
```

Tutorial 8 – Implement Grouping and Ordering

Implement GROUP BY, HAVING and ORDER BY clauses with suitable examples.

Input & Output – GROUP BY example

```
4 rows in set (0.011 sec)

mysql> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std LEFT JOIN MARKS
  ON Std.Student_ID = MARKS.Student_ID
    -> UNION
    -> SELECT Std.Student_ID , Std.Name,MARKS.MARKS FROM Std RIGHT JOIN MARK
S  ON Std.Student_ID = MARKS.Student_ID;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
|        104 | Gayathri  |  NULL |
|       NULL | NULL      |    88 |
+------------+-----------+-------+
5 rows in set (0.299 sec)

mysql> SELECT Student_ID , Name , MARKS
    -> FROM Std
    -> NATURAL JOIN MARKS;
+------------+-----------+-------+
| Student_ID | Name      | MARKS |
+------------+-----------+-------+
|        101 | Harshitha |    86 |
|        102 | Chandana  |    90 |
|        103 | Monisha   |    78 |
+------------+-----------+-------+
3 rows in set (0.300 sec)

mysql> SELECT City ,COUNT(*) AS Total_Students
    -> FROM Std
    -> GROUP BY City;
+-----------+----------------+
| City      | Total_Students |
+-----------+----------------+
| Bangaluru |              2 |
| Mysuru    |              1 |
| Hubli     |              1 |
+-----------+----------------+
3 rows in set (0.311 sec)
```

MySQL Lab Assignment – Input & Output

Tutorial 8 – Implement Grouping and Ordering

Input & Output – GROUP BY with HAVING and ascending ORDER BY

```
mysql> SELECT City ,COUNT(*) AS Total_Students
    -> FROM Std
    -> GROUP BY City;
+-----------+----------------+
| City      | Total_Students |
+-----------+----------------+
| Bangaluru |              2 |
| Mysuru    |              1 |
| Hubli     |              1 |
+-----------+----------------+
3 rows in set (0.311 sec)

mysql> SELECT Subject ,AVG(MARKS) AS Average_MARKS
    -> FROM MARKS
    -> GROUP BY Subject
    -> HAVING AVG(MARKS) > 80;
+---------+---------------+
| Subject | Average_MARKS |
+---------+---------------+
| DBMS    |       85.5000 |
+---------+---------------+
1 row in set (0.342 sec)

mysql> SELECT *
    -> FROM MARKS
    -> ORDER BY MARKS ASC;
+------------+---------+-------+
| Student_ID | Subject | Marks |
+------------+---------+-------+
|        103 | DBMS    |    78 |
|        101 | DBMS    |    86 |
|        105 | DBMS    |    88 |
|        102 | DBMS    |    90 |
+------------+---------+-------+
4 rows in set (0.009 sec)
```

Tutorial 8 – Implement Grouping and Ordering

Input & Output – ORDER BY in ascending and descending order

```
    -> FROM MARKS
    -> GROUP BY Subject
    -> HAVING AVG(MARKS) > 80;
+---------+---------------+
| Subject | Average_MARKS |
+---------+---------------+
| DBMS    |       85.5000 |
+---------+---------------+
1 row in set (0.342 sec)

mysql> SELECT *
    -> FROM MARKS
    -> ORDER BY MARKS ASC;
+------------+---------+-------+
| Student_ID | Subject | Marks |
+------------+---------+-------+
|        103 | DBMS    |    78 |
|        101 | DBMS    |    86 |
|        105 | DBMS    |    88 |
|        102 | DBMS    |    90 |
+------------+---------+-------+
4 rows in set (0.009 sec)

mysql> SELECT *
    -> FROM MARKS
    -> ORDER BY MARKS DESC;
+------------+---------+-------+
| Student_ID | Subject | Marks |
+------------+---------+-------+
|        102 | DBMS    |    90 |
|        105 | DBMS    |    88 |
|        101 | DBMS    |    86 |
|        103 | DBMS    |    78 |
+------------+---------+-------+
4 rows in set (0.011 sec)
```

Tutorial 9 – Implement Procedures

Implement a basic procedure and procedures for INSERT, UPDATE and DELETE operations.

Input & Output – Basic procedure (ShowStudents)

```
4 rows in set (0.011 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE ShowStudents()
    -> BEGIN
    -> SELECT * FROM Std;
    -> END //
Query OK, 0 rows affected (0.554 sec)

mysql> DELIMITER ;
mysql> CALL ShowStudents();
+------------+-----------+-----------+
| Student_ID | Name      | City      |
+------------+-----------+-----------+
|        101 | Harshitha | Bangaluru |
|        102 | Chandana  | Mysuru    |
|        103 | Monisha   | Hubli     |
|        104 | Gayathri  | Bangaluru |
+------------+-----------+-----------+
4 rows in set (0.249 sec)

Query OK, 0 rows affected (0.318 sec)
```

MySQL Lab Assignment – Input & Output

Tutorial 9 – Implement Procedures

Input & Output – INSERT procedure (InsertStd)

```
mysql> CREATE PROCEDURE InsertStd(
    -> IN id INT,
    -> IN Student_name VARCHAR(30),
    -> IN City_name VARCHAR(20))
    -> BEGIN
    -> INSERT INTO Std VALUES (id , Student_name , City_name);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use nea
r '' at line 6
mysql> DELIMITER //
mysql> CREATE PROCEDURE InsertStd(
    -> IN id INT,
    -> IN Student_name VARCHAR(30),
    -> IN City_name VARCHAR(20))
    -> BEGIN
    -> INSERT INTO Std VALUES (id , Student_name , City_name);
    -> END //
Query OK, 0 rows affected (0.067 sec)

mysql> DELIMITER ;
mysql> CALL InsertStd(105,'Chandu','Mangalore');
Query OK, 1 row affected (0.407 sec)

mysql> SELECT * FROM Std;
+------------+-----------+-----------+
| Student_ID | Name      | City      |
+------------+-----------+-----------+
|        101 | Harshitha | Bangaluru |
|        102 | Chandana  | Mysuru    |
|        103 | Monisha   | Hubli     |
|        104 | Gayathri  | Bangaluru |
|        105 | Chandu    | Mangalore |
+------------+-----------+-----------+
5 rows in set (0.014 sec)
```

MySQL Lab Assignment – Input & Output

Tutorial 9 – Implement Procedures

Input & Output – UPDATE procedure (UpdateStd)

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE UpdateStd(
    -> IN id INT,
    -> IN new_City VARCHAR(20))
    -> BEGIN
    -> UPDATE std
    -> SET City = new_City
    -> WHERE Student_ID = id;
    -> END //
Query OK, 0 rows affected (0.533 sec)

mysql> DELIMITER ;
mysql> CALL UpdateStudent(105,'Bangaluru');
ERROR 1305 (42000): PROCEDURE student.UpdateStudent does not exist
mysql> CALL UpdateStd105,'Bangaluru');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use nea
r ','Bangaluru')' at line 1
mysql> CALL UpdateStd(105,'Bangaluru');
Query OK, 1 row affected (0.318 sec)

mysql> SELECT * FROM Std;
+------------+-----------+-----------+
| Student_ID | Name      | City      |
+------------+-----------+-----------+
|        101 | Harshitha | Bangaluru |
|        102 | Chandana  | Mysuru    |
|        103 | Monisha   | Hubli     |
|        104 | Gayathri  | Bangaluru |
|        105 | Chandu    | Bangaluru |
+------------+-----------+-----------+
5 rows in set (0.012 sec)
```

MySQL Lab Assignment – Input & Output

Tutorial 9 – Implement Procedures

Input & Output – DELETE procedure (DeleteStd)

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE DeleteStd(
    -> IN id INT
    -> )
    -> BEGIN
    -> DELETE FROM Std
    -> WHERE Student_ID = id;
    -> END //
Query OK, 0 rows affected (0.324 sec)

mysql> DELIMITER;
    -> ^C
mysql> DELIMITER ;
mysql> CALL DeleteStd(105);
Query OK, 1 row affected (0.274 sec)

mysql> SELECT * FROM Std;
+------------+-----------+-----------+
| Student_ID | Name      | City      |
+------------+-----------+-----------+
|        101 | Harshitha | Bangaluru |
|        102 | Chandana  | Mysuru    |
|        103 | Monisha   | Hubli     |
|        104 | Gayathri  | Bangaluru |
+------------+-----------+-----------+
4 rows in set (0.011 sec)
```

Tutorial 10 – Implement Triggers

Implement validation/logging triggers before and after INSERT, DELETE and UPDATE operations.

Input & Output – BEFORE INSERT validation trigger

```
mysql> CREATE TABLE STUDENT_LOG(
    -> log_ID INT AUTO_INCREMENT PRIMARY KEY,
    -> Student_ID INT,
    -> Action VARCHAR(30),
    -> Log_Time TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
    -> DELIMITER //
ERROR 1050 (42S01): Table 'student_log' already exists
mysql> CREATE TRIGGER Before_Student_Insert
    -> BEFORE INSERT ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> IF NEW.Name = '' THEN
    -> SET NEW.Name = 'Unknown';
    -> END IF;
    -> END //
Query OK, 0 rows affected (0.362 sec)

mysql> DELIMITER ;
mysql> INSERT INTO Std
    -> VALUES (106,'','Banagluru');
Query OK, 1 row affected (0.297 sec)

mysql> SELECT * FROM Student;
ERROR 1146 (42S02): Table 'student.student' doesn't exist
mysql> SELECT * FROM Std;
+------------+-----------+-----------+
| Student_ID | Name      | City      |
+------------+-----------+-----------+
|        101 | Harshitha | Bangaluru |
|        102 | Chandana  | Mysuru    |
|        103 | Monisha   | Hubli     |
|        104 | Gayathri  | Bangaluru |
|        106 | Unknown   | Banagluru |
+------------+-----------+-----------+
5 rows in set (0.009 sec)
```

MySQL Lab Assignment – Input & Output

Tutorial 10 – Implement Triggers

Input & Output – AFTER INSERT trigger

```
mysql> DELIMITER //
mysql> CREATE TRIGGER After_Student_Delete
    -> AFTER DELETE ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> ^C
mysql> DELIMITER //
mysql> CREATE TRIGGER After_Student_Insert
    -> AFTER INSERT ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> INSERT INTO Student_LOG(Student_ID , Action)
    -> VALUES (NEW.Student_ID,'INSERT');
    -> END //
Query OK, 0 rows affected (0.315 sec)

mysql> DELIMITER ;
mysql> SELECT * FROM Student_LOG;
Empty set (0.008 sec)
```

Tutorial 10 – Implement Triggers

Input & Output – BEFORE UPDATE validation and AFTER UPDATE logging

```
mysql> DELIMITER //
mysql> CREATE TRIGGER Before_Student_Update
    -> BEFORE UPDATE ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> IF NEW.City = '' THEN
    -> SET NEW.City = OLD.City;
    -> END IF;
    -> END //
Query OK, 0 rows affected (0.328 sec)

mysql> DELIMITER ;
mysql> DELIMITER //
mysql> CREATE TRIGGER After_Student_Update
    -> AFTER UPDATE ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> INSERT INTO Student_LOG(Student_ID , Action)
    -> VALUES (NEW.Student_ID,'UPDATE');
    -> END //
Query OK, 0 rows affected (0.306 sec)

mysql> DELIMITER ;
mysql> UPDATE Std
    -> SET City = 'Mysuru'
    -> WHERE Student_ID = 101;
Query OK, 1 row affected (0.297 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Student_LOG;
+--------+------------+--------+---------------------+
| log_ID | Student_ID | Action | Log_Time            |
+--------+------------+--------+---------------------+
|      1 |        101 | UPDATE | 2026-10-08 21:38:15 |
+--------+------------+--------+---------------------+
1 row in set (0.235 sec)
```

Tutorial 10 – Implement Triggers

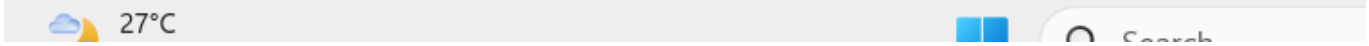
Input & Output – BEFORE DELETE and AFTER DELETE logging

```
mysql> DELIMITER //
mysql> CREATE TRIGGER Before_Student_Delete
    -> BEFORE DELETE ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> INSERT INTO Student_LOG(Student_ID,Action)
    -> VALUES(OLD.Student_ID,'BEFORE DELETE');
    -> END //
Query OK, 0 rows affected (0.329 sec)

mysql> DELIMITER ;
mysql> DELIMITER //
mysql> CREATE TRIGGER After_Student_Delete
    -> AFTER DELETE ON Std
    -> FOR EACH ROW
    -> BEGIN
    -> INSERT INTO Student_LOG(Student_ID,Action)
    -> VALUES(OLD.Student_ID,'AFTER DELETE');
    -> END //
Query OK, 0 rows affected (0.095 sec)

mysql> DELIMITER ;
mysql> DELETE FROM Std
    -> WHERE Student
    -> _ID =106;
ERROR 1064 (42000): You have an error in your SQL syntax; chec
hat corresponds to your MySQL server version for the right syn
r '_ID =106' at line 3
mysql> DELETE FROM Std
    -> WHERE Student_ID = 106;
Query OK, 1 row affected (0.289 sec)

mysql> SELECT * FROM Student_LOG;
+--------+------------+---------------+---------------------+
| log_ID | Student_ID | Action        | Log_Time            |
+--------+------------+---------------+---------------------+
|      1 |        101 | UPDATE        | 2026-10-08 21:38:15 |
|      2 |        106 | BEFORE DELETE | 2026-10-08 21:46:18 |
|      3 |        106 | AFTER DELETE  | 2026-10-08 21:46:18 |
+--------+------------+---------------+---------------------+
3 rows in set (0.245 sec)
```



MySQL Lab Assignment – Input & Output